
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 13

O Pré-processador em C

Exercícios de Autorrevisão (13.1–13.3)

Resolvidos passo a passo, abordagem incremental
Capítulos 1 a 13

Construindo sobre os Capítulos Anteriores

No Capítulo 13, voltamos ao `#include` e `#define` que usamos desde o Cap. 2 – agora com profundidade total. O **pré-processador** é um programa que roda *antes* do compilador, modificando o código-fonte:

- **Diretivas** – linhas que começam com `#`, processadas *antes* da compilação
- **Constantes simbólicas** – `#define` para nomes legíveis no lugar de valores literais
- **Macros** – substituições com parâmetros, parecidas com funções mas expandidas em linha
- **Inclusão de arquivos** – `#include <...>` (sistema) e `#include "..."` (usuário)
- **Compilação condicional** – `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, `#endif`
- **Operadores de pré-processamento** – `#` (*stringize*) e `##` (*token paste*)
- **Constantes predefinidas** – `__LINE__`, `__FILE__`, `__DATE__`, `__TIME__`, `__STDC__`
- `assert` – macro de diagnóstico para depuração

Boa prática de programação

Macros são poderosas mas perigosas. Sempre que possível, prefira: `const` a `#define` para constantes; `inline` (C99+) a macros para pequenas funções. Reserve macros para o que só elas podem fazer: compilação condicional, geração de código, debug.

Contents

1	Exercício 13.1 – Preenchimento de Lacunas (13 itens)	2
2	Exercício 13.2 – Constantes Predefinidas	3
3	Exercício 13.3 – Diretivas do Pré-processador	4

1. Exercício 13.1 – Preenchimento de Lacunas (13 itens)

Resposta detalhada

- a) Toda diretiva do pré-processador começa com #.
- b) Compilação condicional estendida usa as diretivas `#elif` e `#else`.
- c) A diretiva `#define` cria macros e constantes simbólicas.
- d) Apenas caracteres de **espaço em branco** podem aparecer antes de uma diretiva na mesma linha.
- e) A diretiva `#undef` descarta nomes de constantes simbólicas e macros.
- f) As notações abreviadas para `#if defined(nome)` e `#if !defined(nome)` são `#ifdef` e `#ifndef`.
- g) A **compilação condicional** permite controlar a execução das diretivas e a compilação do código.
- h) A macro `assert` imprime mensagem e termina a execução se a expressão for 0.
- i) A diretiva `#include` insere o conteúdo de um arquivo em outro.
- j) O operador `##` (*token paste*) concatena seus dois argumentos.
- k) O operador `#` (*stringize*) converte seu operando em string.
- l) O caractere `\` (barra invertida) indica que o texto de substituição continua na próxima linha.
- m) A diretiva `#line` faz com que as linhas sejam renumeradas a partir do valor indicado.

Resumo das diretivas:

Diretiva	Finalidade
<code>#include</code>	inclui conteúdo de arquivo
<code>#define</code>	define constante ou macro
<code>#undef</code>	remove definição
<code>#ifdef</code> / <code>#ifndef</code>	teste de definição
<code>#if</code> / <code>#elif</code> / <code>#else</code> / <code>#endif</code>	compilação condicional
<code>#error</code>	gera erro de compilação
<code>#pragma</code>	instrução específica do compilador
<code>#line</code>	renumera linhas

2. Exercício 13.2 – Constantes Predefinidas

Resposta detalhada

```
1  /* Ex13.2: macros_predefinidas.c
2     Imprime os valores das constantes simbólicas
3     predefinidas */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      printf( "__LINE__ = %d\n", __LINE__ );
10     printf( "__FILE__ = %s\n", __FILE__ );
11     printf( "__DATE__ = %s\n", __DATE__ );
12     printf( "__TIME__ = %s\n", __TIME__ );
13     printf( "__STDC__ = %d\n", __STDC__ );
14
15     return 0;
16 }
```

Saída típica:

Saída do programa

```
__LINE__ = 5
__FILE__ = macros_predefinidas.c
__DATE__ = May 13 2026
__TIME__ = 14:23:45
__STDC__ = 1
```

Significado de cada constante:

- `__LINE__` – número da linha atual no código-fonte (inteiro).
- `__FILE__` – nome do arquivo-fonte atual (string).
- `__DATE__` – data da compilação (string, formato “Mmm dd yyyy”).
- `__TIME__` – hora da compilação (string, formato “hh:mm:ss”).
- `__STDC__` – vale 1 se compilador adere ao padrão ANSI/ISO C.

Aplicações práticas:

- **Logs com origem:** `printf("Erro em %s:%d", __FILE__, __LINE__)`.
- **Identificação de build:** mostrar data/hora da compilação no `-version` do programa.
- **Portabilidade:** checar `__STDC__` antes de usar recursos não-portáteis.

Boa prática de programação

Essas macros são a base da macro `assert` e dos sistemas de log em C. Sempre que precisar identificar onde algo ocorreu no código, lembre-se de `__FILE__` e `__LINE__`.

3. Exercício 13.3 – Diretivas do Pré-processador

Resposta detalhada

a) Definir YES como 1:

```
1 #define YES 1
```

b) Definir NO como 0:

```
1 #define NO 0
```

c) Incluir common.h (mesmo diretório):

```
1 #include "common.h"
```

Diferença crucial:

- `#include "arquivo.h"` – procura primeiro no diretório do arquivo-fonte (cabecinhos do projeto).
- `#include <arquivo.h>` – procura nos diretórios do sistema (cabecinhos da biblioteca-padrão).

d) Renumerar a partir de 3000:

```
1 #line 3000
```

Útil para geradores de código que produzem arquivos C, para que mensagens de erro do compilador apontem para o arquivo *original* (não o gerado).

e) Redefinir TRUE se estiver definida (sem `#ifndef`):

```
1 #if defined( TRUE )
2     #undef TRUE
3     #define TRUE 1
4 #endif
```

f) Redefinir TRUE se definida (com `#ifndef`):

```
1 #ifndef TRUE
2     #undef TRUE
3     #define TRUE 1
4 #endif
```

g) Definir FALSE dependendo de TRUE:

```
1 #if TRUE
2     #define FALSE 0
3 #else
4     #define FALSE 1
5 #endif
```

h) Macro VOLUME_CUBO:

```
1 #define VOLUME_CUBO( x ) ( ( x ) * ( x ) * ( x ) )
```

Erro comum de programação

Parênteses são essenciais em macros! Sem eles:

```
1 #define VOLUME_CUBO( x )  x * x * x  /*  
   PERIGOSO */  
2  
3 VOLUME_CUBO( 1 + 2 )  /* expande para: 1 + 2 *  
   1 + 2 * 1 + 2 = 7 */  
4  
   /* deveria ser: 3 * 3  
   * 3 = 27      */
```

Com parênteses corretos:

```
1 VOLUME_CUBO( 1 + 2 )  /* expande para: (1+2)  
   *(1+2)*(1+2) = 27 */
```

Regra de ouro: sempre envolver cada parâmetro e a expressão inteira em parênteses.

Gabarito Rápido – Capítulo 13: Autorrevisão

Respostas Resumidas

13.1:

- a) # b) #elif, #else c) #define
- d) espaço em branco e) #undef f) #ifdef, #ifndef
- g) compilação condicional h) assert i) #include
- j) ## k) # l) \ m) #line

13.2: Programa imprime `__LINE__`, `__FILE__`, `__DATE__`, `__TIME__`, `__STDC__` – todas as constantes predefinidas pelo padrão.

13.3 – 8 diretivas:

- a) `#define YES 1`
- b) `#define NO 0`
- c) `#include "common.h"`
- d) `#line 3000`
- e) `#if defined(TRUE) ... #undef ... #define ... #endif`
- f) `#ifdef TRUE ... #undef ... #define ... #endif`
- g) `#if TRUE ... #else ... #endif`
- h) `#define VOLUME_CUBO(x) ((x)*(x)*(x))`